Lecture 9 Notes

The Transport Layer: Multiplexing, Ports, UDP, and the TCP Connection

Computer Networks

Learning outcomes

- **[ ]** Explain why the transport layer is needed between the application layer and network layer.
- **[ ]** Use the terms port, socket, multiplexing, demultiplexing, and 4-tuple correctly.
- **[ ]** Compare UDP and TCP using speed, reliability, ordering, and connection setup.
- **[ ]** Trace the TCP 3-way handshake and the TCP connection teardown.
- **[ ]** Choose UDP or TCP for a given application scenario and justify the choice.

1. Big picture: what Layer 4 does

In the previous part of the course, we studied how application-layer protocols such as DNS, HTTP, and email create messages. The next question is: **how does an application message travel to the correct application on another machine?**

Layer view	Main job	Example question it answers
Application layer	Creates meaningful messages for the user or app.	What does this HTTP request, DNS query, or email mean?
Transport layer	Provides process-to-process delivery using ports. It turns application data into transport segments.	Which process or app should receive this data?
Network layer	Provides host-to-host delivery using IP addresses.	Which computer should this packet reach?

> **Note**
> A packet reaching the correct laptop is not enough. Your laptop may have a browser, video call, messaging app, and music app running together. The operating system still needs a way to decide which process should receive each packet. That is why ports are needed.

Quick concept map

Application message -> TCP/UDP segment -> IP packet -> data-link frame

- TCP and UDP run in end systems: the sender and receiver. Regular forwarding routers mainly use network-layer information for routing.

- The transport header contains port information. TCP also contains reliability-related fields such as sequence numbers, acknowledgment numbers, flags, and window size.

> **Pause and check 1**
> Your laptop uses one WiFi IP address, but Spotify, Chrome, and WhatsApp Web are running at the same time. What extra number helps the operating system deliver received data to the correct app? Answer: the port number.

2. Ports, sockets, and the 4-tuple

Think of a computer like a large apartment building. The **IP address** is like the street address of the building. The **port number** is like the apartment number. You need both to deliver the message to the right place.

Term	Meaning	Example
Port number	A 16-bit number used by TCP or UDP to identify a service or application process on a host.	443 for HTTPS, 53 for DNS, 80 for HTTP
Socket endpoint	One communication endpoint: IP address plus port number.	142.250.183.206:443
TCP connection 4-tuple	The unique identity of one TCP conversation: source IP, source port, destination IP, destination port.	192.168.1.8:53120 -> 142.250.183.206:443
Ephemeral port	A temporary client-side port selected for one outgoing connection.	53120, 51514, or another high port

Common port ranges

Range	Name	Typical use
0-1023	System or well-known ports	Common services such as SSH 22, DNS 53, HTTP 80, HTTPS 443
1024-49151	User or registered ports	Registered applications and services

49152-65535	Dynamic or private ports	Often used as temporary client-side ports

Note
A server usually listens on a well-known port, such as 443 for HTTPS. A client usually uses a temporary port. This is why thousands of clients can connect to the same server port without confusion.

Pause and check 2
In the connection 192.168.1.8:53120 -> 142.250.183.206:443, which port belongs to the server? Which port is likely temporary on the client? Answer: server port 443; client temporary port 53120.

3. Multiplexing and demultiplexing

Many applications may send data at the same time. The transport layer must combine outgoing data streams and separate incoming data streams. These two actions are called **multiplexing** and **demultiplexing**.

Action	Where it happens	What happens
Multiplexing	Sender side	The transport layer collects data from multiple application processes, adds TCP/UDP headers, and sends segments into the network.
Demultiplexing	Receiver side	The transport layer reads port information and delivers the data to the correct socket or application process.

Example
Imagine three applications on the same laptop: browser traffic to port 443, messaging traffic to port 5222, and email traffic to port 587. The sender multiplexes these streams. The receiver demultiplexes incoming segments using ports and socket information.

How the receiver decides

- For UDP, the operating system mainly uses the destination IP address, destination port, and protocol to choose the receiving socket.

- For TCP, each connection is identified by the 4-tuple. This is the key reason one web server can maintain many simultaneous connections on port 443.

Pause and check 3
Two browser tabs both connect to the same server on port 443. How can the server keep them separate? Answer: each TCP connection has a different 4-tuple, usually because the client uses different temporary source ports.

4. UDP: simple, fast, and connectionless

UDP stands for User Datagram Protocol. It is a lightweight transport protocol. UDP does not establish a connection before sending data. It sends datagrams quickly, but it does not promise delivery, ordering, or retransmission at the transport layer.

UDP header field	Purpose
Source port	Identifies the sending process when a reply is needed.
Destination port	Identifies the receiving process or service.
Length	Length of the UDP datagram, including UDP header and data. The minimum is 8 bytes.
Checksum	Helps detect corruption or misdelivery.

What UDP gives and does not give

Feature	UDP behaviour	Student interpretation
Connection setup	No handshake	Data can be sent immediately.
Reliability	No delivery guarantee	A lost datagram may simply be lost unless the application handles it.
Ordering	No ordering guarantee	Datagrams may arrive out of order.
Overhead	Small header and simple behaviour	Useful when low latency is more important than perfect delivery.

Application control	The application can decide how to handle loss	Games, calls, and live video can skip old data instead of waiting.

Note
UDP is not "bad TCP". UDP is useful when the application prefers speed and timing over built-in transport reliability. The application may add its own recovery rules if needed.

Where UDP is often useful

- DNS lookups: small request-response exchanges where speed matters. If needed, the application can retry.
- Live audio or video: late packets may be less useful than skipped packets because users notice lag.
- Online games: frequent position updates can tolerate some loss because newer updates replace older ones.
- QUIC and HTTP/3: modern web transport can use UDP while adding reliability, security, and multiplexing at a higher layer.

Pause and check 4
In a live cricket stream, which is usually worse: losing one video frame or pausing the whole stream to wait for a retransmission? Usually, pausing is worse because it causes visible lag.

Optional self-practice
Netcat command syntax may vary by operating system, but the idea is: start one terminal listening with UDP and another terminal sending UDP messages.

Receiver terminal: nc -ul 5000
Sender terminal: nc -u SERVER_IP 5000

5. TCP: connection-oriented and reliable
TCP stands for Transmission Control Protocol. TCP is designed for applications that need reliable, in-order delivery of a byte stream. Before normal data transfer begins, TCP establishes a connection.

TCP promise	**Meaning for the application**
Reliable delivery	TCP tries to detect lost data and retransmit it.
In-order delivery	The application receives the byte stream in the correct order.

Error checking	Checksums help detect corrupted segments.
Flow control	The receiver can tell the sender how much buffer space is available.
Congestion control	TCP adjusts sending behaviour when the network appears congested.

The TCP 3-way handshake

The handshake is the polite start of a TCP conversation. It lets both sides agree that they are ready and exchange initial sequence numbers.

Step	Message	Direction	Meaning
1	SYN	Client to server	Client asks to start a connection and sends its initial sequence number.
2	SYN-ACK	Server to client	Server acknowledges the client and sends its own initial sequence number.
3	ACK	Client to server	Client acknowledges the server. The TCP connection is now established.

Why not only two messages?

With three messages, both sides can confirm that sending and receiving works in both directions, and both sides know each other's initial sequence number.

TCP connection teardown

TCP also has a polite goodbye. A common close uses FIN and ACK messages:

- Client sends FIN: "I have no more data to send."
- Server sends ACK: "I received your FIN."
- Server later sends FIN: "I am done too."
- Client sends final ACK: "I received your FIN."

Sequence numbers and acknowledgments

TCP treats application data as a stream of bytes. Sequence numbers identify byte positions in that stream. Acknowledgement numbers tell the sender the next byte expected by the receiver. If data is lost, TCP can retransmit it.

> **Phone-call analogy**
> When you talk on a phone call, you listen for signals like "yes" or "I heard you". TCP ACKs serve a similar role: they confirm what has been received and what is expected next.

Important TCP fields to recognise

Important TCP field	Why students should care
Source and destination ports	Identify the sending and receiving applications.
Sequence number	Helps place data in order and detect missing bytes.
Acknowledgment number	Tells the sender the next expected byte.
Flags such as SYN, ACK, FIN	Control connection setup, data acknowledgement, and connection closing.
Window size	Supports flow control by advertising available receiver buffer space.
Checksum	Helps detect corrupted data.

> **Pause and check 5**
> A receiver gets segment 3 before segment 2. What should TCP do for the application? Answer: TCP waits/reorders so the application receives data in the correct order.

6. Choosing UDP or TCP

When selecting a transport protocol, do not begin by memorising app names. Begin by asking what the application needs most.

Application requirement	Better fit	Reason

Every byte must arrive correctly and in order.	TCP	Examples: file download, email, login form, bank transaction.
Low latency matters more than perfect delivery.	UDP	Examples: live audio, live video, online games, telemetry.
Small request and small response; retry is acceptable.	UDP	Example: typical DNS lookup.
Web traffic using HTTP/1.1 or HTTP/2.	TCP	These versions traditionally run over TCP with TLS for HTTPS.
Web traffic using HTTP/3.	QUIC over UDP	QUIC uses UDP but adds streams, security, loss recovery, and congestion control above UDP.

Quick app examples

Application	**Common transport choice**	**Why**
DNS lookup	UDP	Fast query-response; retry can happen if no response arrives.
Email sending	TCP	Messages must not lose bytes.
File transfer	TCP	File bytes must arrive completely and in order.
Live video call	UDP or UDP-based transport	Late audio/video is often useless; smooth timing matters.
HTTPS website	TCP for HTTP/1.1 or HTTP/2; QUIC/UDP for HTTP/3	Depends on HTTP version and browser/server support.

Caution
Real systems can be mixed. For example, an app may use TCP for login and payment, but UDP or QUIC for real-time media. Always justify the choice using the application requirement: reliability, ordering, latency, and control.

7. Hands-on revision commands

Use these commands only in a safe lab environment or on your own machine. The goal is to connect the theory to what your operating system is doing.

Goal	Command or tool	What to observe
List active TCP connections	ss -tn or netstat -tn	Local IP:port, remote IP:port, and connection state.
See a TCP handshake	Wireshark filter: tcp.flags.syn == 1	SYN, SYN-ACK, ACK; ports; sequence numbers.
Open an HTTPS connection	curl -v https://example.com	Connection setup and TLS after TCP.
Try UDP chat	nc -ul 5000 and nc -u SERVER_IP 5000	UDP messages without a TCP-style handshake.

8. Ten-minute self-check

Cover the answers first, try each question, then check your understanding.

1. 1. Network layer delivery is host-to-host. Transport layer delivery is _____-to-_____.
2. 2. A socket endpoint is made from _____ + _____.
3. 3. The TCP connection identity is the 4-tuple: source IP, source port, destination IP, and _____.
4. 4. UDP starts data transfer with a 3-way handshake. True or false?
5. 5. Write the three TCP handshake messages in order.
6. 6. In TCP, an ACK number usually tells the sender the next _____ expected.
7. 7. Which protocol is usually better for a file download where every byte must arrive?
8. 8. Which port is commonly used for HTTPS?

9. Scenario-based questions for after-class practice

Instructions: Attempt each scenario in your own words first. In exams and interviews, marks usually come from the justification, not just writing "TCP" or "UDP".

Scenario 1: Live cricket streaming app

A company is building a live cricket streaming app. The app has three parts:

- Live video and audio during the match.
- Scorecard updates every few seconds.
- User login, subscription payment, and downloading match highlights after the match.

Question: For each part, choose a suitable transport approach: TCP, UDP, or a UDP-based transport such as QUIC. Explain your choice using latency, reliability, ordering, and packet loss.

_____
_____
_____
_____
_____
_____
_____
_____
_____
_____

Scenario 2: One HTTPS server, many students

A college web server is listening on 203.0.113.20:443. During lab time, three connections arrive:

Connection	Source endpoint	Destination endpoint
A	10.0.0.12:51514	203.0.113.20:443
B	10.0.0.12:51515	203.0.113.20:443
C	10.0.0.25:51514	203.0.113.20:443

Question: All three connections use the same server IP and server port. Explain how the server keeps them separate. Also write the TCP handshake messages and explain what TCP does if a data segment is lost.

_____
_____
_____
_____
_____
_____
_____
_____
_____
_____

Final key takeaways

[ ] Transport layer delivery is process-to-process. IP gets data to a host; ports get it to the correct application process.

[ ] A socket endpoint is IP address plus port. A TCP connection is identified by a 4-tuple.

[ ] Multiplexing happens at the sender; demultiplexing happens at the receiver.

[ ] UDP is connectionless and low overhead, but it does not guarantee delivery or ordering.

[ ] TCP is connection-oriented and reliable. It uses a 3-way handshake, sequence numbers, ACKs, and retransmission.

[ ] Protocol choice depends on application needs: reliability/order vs latency/control.

References and further reading

- Course PPT: The Transport Layer - Multiplexing, Ports, UDP, and the TCP Connection.
- J. Kurose and K. Ross, Computer Networking: A Top-Down Approach, 8th ed., Sections 3.1-3.3 and 3.5.
- RFC 768, User Datagram Protocol, RFC Editor/IETF.
- RFC 9293, Transmission Control Protocol (TCP), RFC Editor/IETF.
- IANA, Service Name and Transport Protocol Port Number Registry.
- RFC 9000, QUIC: A UDP-Based Multiplexed and Secure Transport, RFC Editor/IETF.
- RFC 9114, HTTP/3, RFC Editor/IETF.
- Wireshark documentation and ss/netstat manual pages for packet and connection inspection.